Experiment 4

Title: Real-time Event Processing System using Queue Data Structure

Objectives:

- To design and implement a real-time event processing system.
- To utilize a Queue data structure for managing incoming events.
- To enable adding new events to the queue.
- To facilitate processing and removal of the longest-waiting event.
- To provide functionality for displaying all pending events.
- To allow cancellation of events before processing.
  Problem Statement:
  Efficient and timely handling of events in a real-time system is crucial. The challenge lies in managing the sequence of events, ensuring oldest events are processed first, providing visibility into pending events, and allowing for event cancellation, all while maintaining system responsiveness.

Outcomes:

- A functional real-time event processing system capable of handling event queues.
- Implementation of add, process, display, and cancel event operations.
- Demonstrable use of a Queue data structure for First-In, First-Out (FIFO) event management.

Software and Hardware Requirements:

- **Software:**
- Operating System: Windows 10/11, macOS, or Linux distribution.
- Programming Language: Python 3.x (or Java, C++, etc., depending on chosen implementation).
- Development Environment: VS Code, PyCharm, IntelliJ IDEA, or similar IDE.

Hardware:

- Processor: Intel Core i3/AMD Ryzen 3 or higher.
- RAM: 4GB or more.
- Storage: Minimum 100MB free space for project files.
  Date of Completion: [Insert Date of Completion]

Theory Concept in brief:

A Queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. Events are added to the rear (enqueue) and removed from the front (dequeue). This structure ensures that events are processed in the order they are received, which is essential for real-time systems where chronological processing is often required.

Algorithm:

- **Initialize Queue:** Create an empty queue to store events.
- **Add Event:**
  - Take event details as input.
  - Enqueue the event into the queue.
- **Process Next Event:**
  - If the queue is not empty, dequeue the event from the front.
  - Perform processing logic for the dequeued event.
- **Display Pending Events:**
  - Iterate through the queue from front to rear.
  - Print details of each event in the queue.
- **Cancel an Event:**
  - Take the event identifier to be canceled as input.
  - Search for the event in the queue.
  - If found and not yet processed (i.e., not at the front if processing is immediate upon dequeue), remove the event from its current position in the queue. This might require re-constructing the queue or using a data structure that supports efficient deletion from the middle.

Flowchart:
[A detailed flowchart depicting the main functionalities (Add, Process, Display, Cancel) and their decision points and process flows would be included here.]

Test Cases:

- **Add Multiple Events:** Add events A, B, C, D to the queue.
- **Process Events:** Process the next two events (A, B).
- **Display Pending Events:** Verify that C and D are displayed.
- **Cancel Event:** Cancel event C.
- **Display Pending Events (after cancellation):** Verify that only D is displayed.

- **Process Remaining Event:** Process the next event (D).
- **Display Pending Events (empty queue):** Verify that no events are displayed.
- **Attempt to Process Empty Queue:** Verify graceful handling.
- **Attempt to Cancel Non-Existent Event:** Verify graceful handling.

Test Data Set (if applicable):

- Event IDs: E001, E002, E003, E004
- Event Descriptions: "User Login", "Database Update", "File Upload", "API Request"
- Event Timestamps: (simulated or actual timestamps)

Mathematical Model (if applicable):
The system implicitly models a queuing system where events arrive and are served. Performance metrics such as average waiting time, queue length, and throughput could be analyzed using queuing theory, but for this basic implementation, a formal mathematical model is not strictly required.

Conclusion/Analysis of Implementation:

The implementation successfully demonstrates a real-time event processing system using a Queue data structure. The FIFO nature of the queue ensures proper event ordering, and the provided functionalities address the core requirements of adding, processing, displaying, and canceling events. The system provides a foundational framework for more complex real-time event management scenarios. Further enhancements could include priority queues, event logging, and distributed event handling.